Demystifying Recursion

In this section, we will uncover the inner workings of recursive function calls, revealing the underlying system choreography. We'll explore how recursive calls are tracked using the call stack and how to represent recursive calls visually using recursion trees

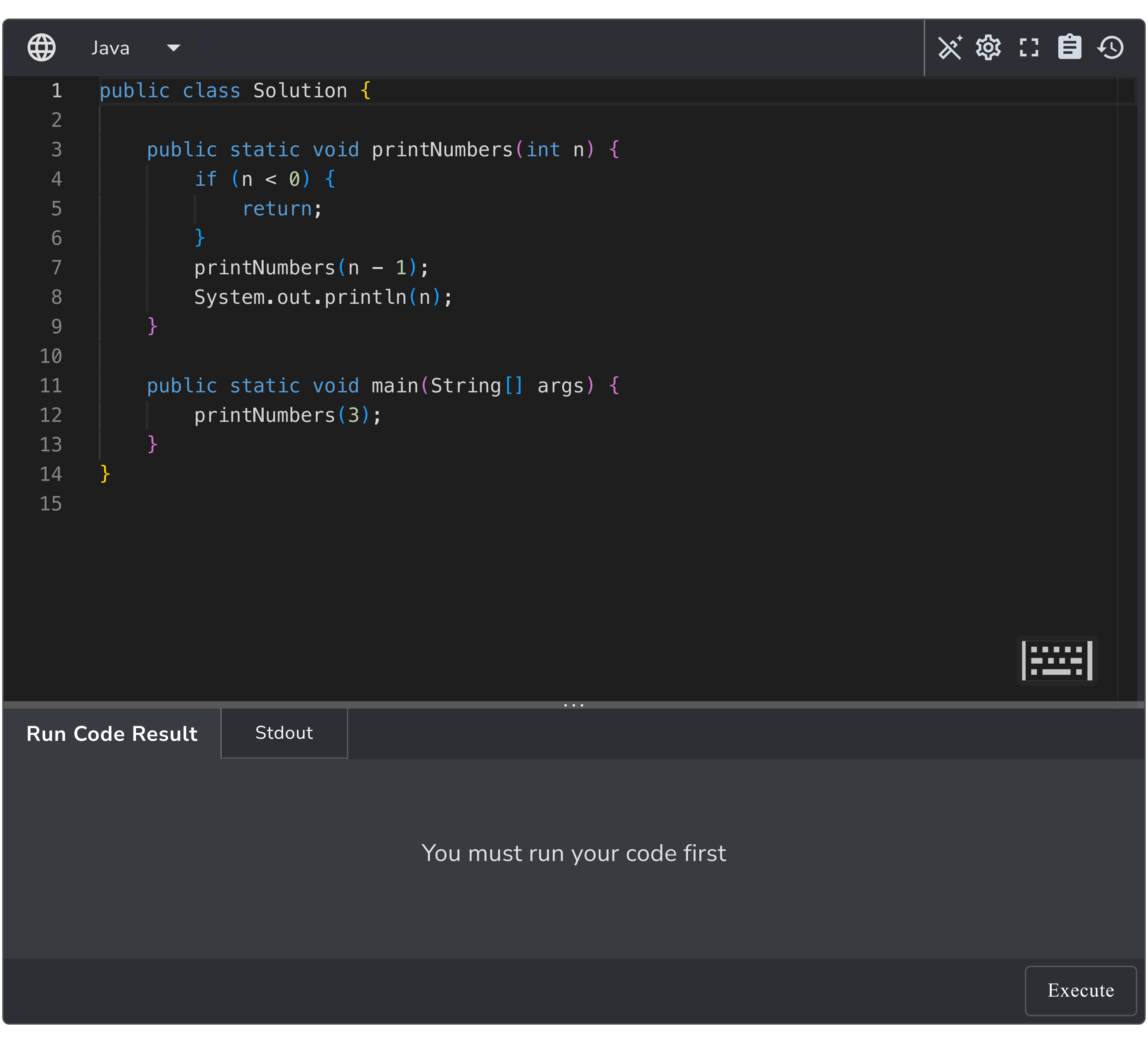
Let's start with understanding how we can visualize the execution of a recursive function using a call tree.

Recursion tree representation

A recursion tree is a useful way to visualize the flow of a recursive function. Let's consider a pseudocode for the `printNumbers()` function that recursively prints numbers from `0` to `n` in non-decreasing order:

```
function printNumbers(n) {
  if (n < 0) {
    return
  }
  printNumbers(n-1)
  print(n)
}
```

Here is the full working code with driver calling `printNumbers()` function with `3` as argument to parameter `n`:



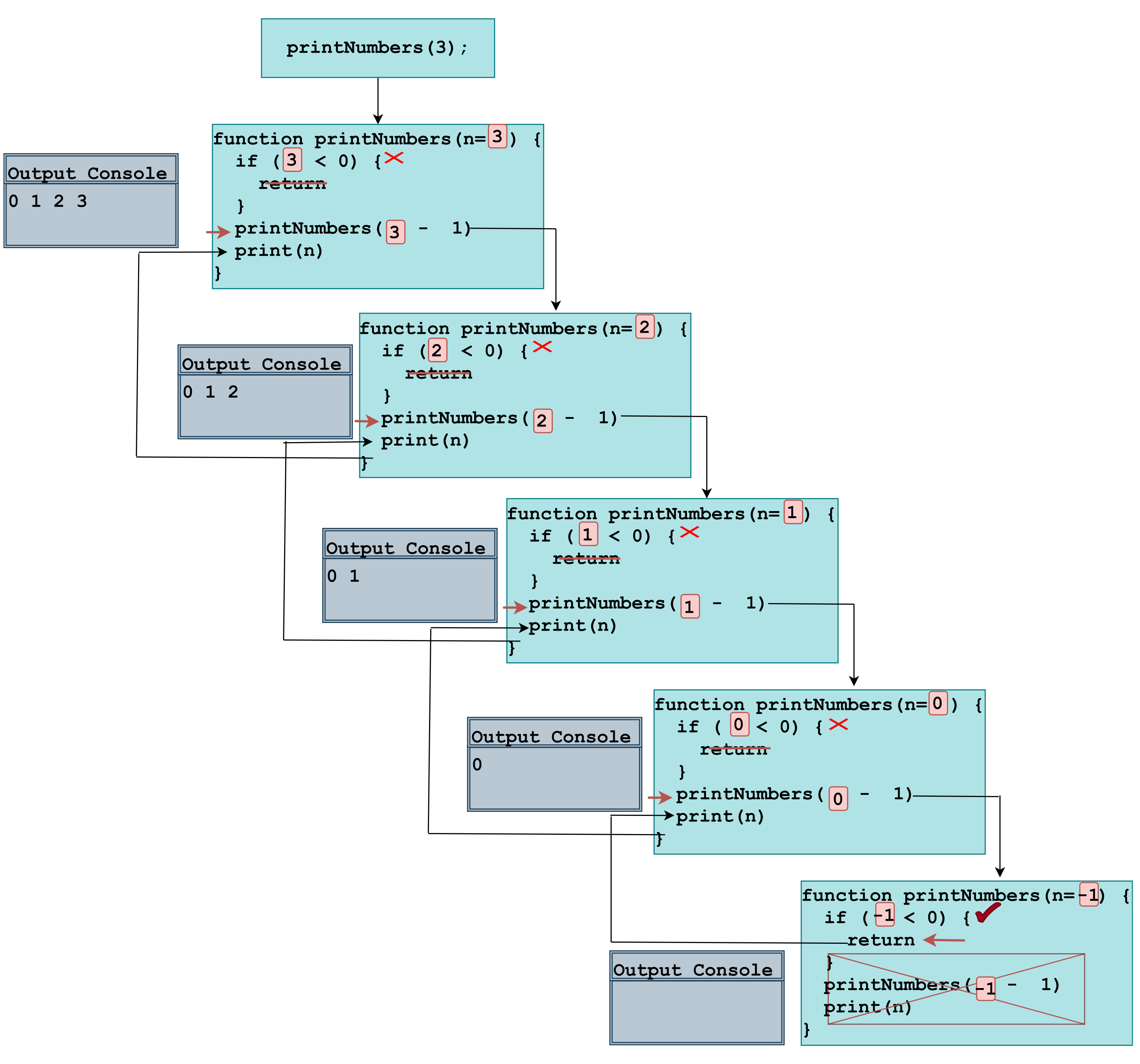
```java
public class Solution {

    public static void printNumbers(int n) {
        if (n < 0) {
            return;
        }
        printNumbers(n - 1);
        System.out.println(n);
    }

    public static void main(String[] args) {
        printNumbers(3);
    }
}
```

You must run your code first

Let's visualize the call `printNumbers(3)` as a recursion tree:



Note that in the above tree diagram mentioned, the first call to the recursive function is represented at the root. Each node represents a separate instance of the same function, but called with different a argument.

The execution control branches to the bottom of the tree until it encounters a terminal/ leaf. After that, a bottom-up phase executes epilogues which print the `n`'s value at the particular call.

Here is another way to represent the same recursive call using the tree representation:

```
printNumbers(3)
|
|--- printNumbers(2)
|     |
|     |--- printNumbers(1)
|     |     |
|     |     |--- printNumbers(0)
|     |     |
|     |     |--- print (1)
|     |
|     |--- print (2)
|
|--- print (3)
```

Challenge: Draw a tree representation for a call `printNums(3)` but suppose that the function has the `print(n)` statement in the prologue section instead of the epilogue.

Call stack representation

The tree representation is easy to understand, but its abstraction is a bit far from what a recursive program does in memory while executing in the computer memory. Call stack representation is one step closer to the actual handling of the recursive calls management done by the program's call stack.

As we know, a call stack is a stack-based data structure used by each executing program. It stores information to handle the execution flow smoothly. Whenever the program invokes a function or sub-routine, it pushes a new stack frame over the top.

Here, a stack frame contains the execution context (e.g., parameters, arguments, autos, etc.) of the currently executing function/ subroutine and the address to return when it terminates. When a function completes its execution, it causes its stack frame to get popped off the stack.

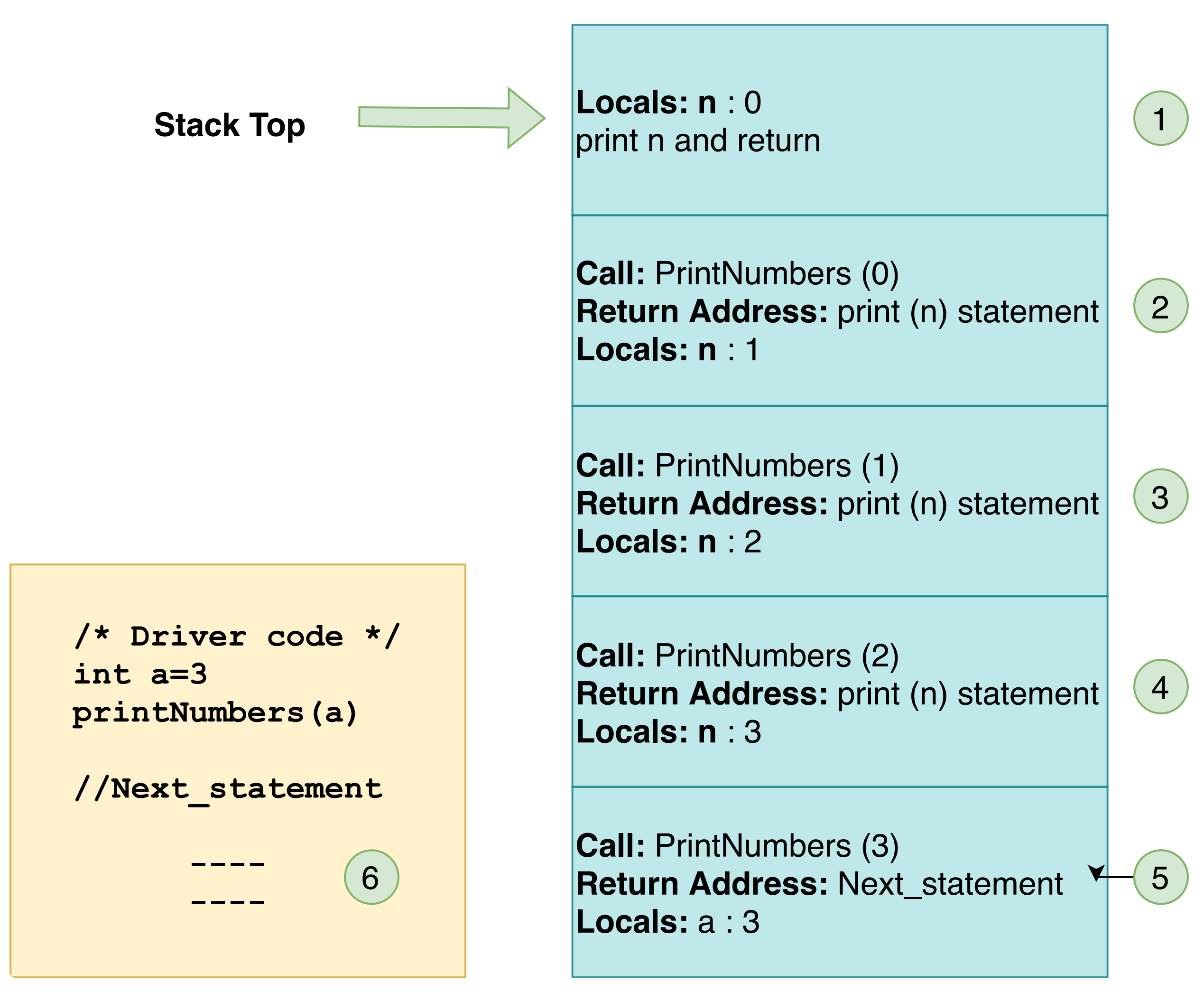
Assume a variation of the recursive function to print numbers from `0` to `n`.

```
function printNumbers(n) {
  if (n > 0) {
    printNumbers(n-1)
  }
  print(n)
}
```

To visualize the execution of the `printNumbers`, imagine a physical stack of books. As we add books (representing function calls), the stack grows. When we remove books (representing completed function calls), the stack shrinks. Here is how the `printNumbers(3)` call be represented using a call stack:

1. Initially, `printNumbers(3)` is called. The state of the function, including the value `3`, is stored in the first frame, and placed onto the call stack.
2. The function calls itself with `n-1`, which results in `printNumbers(2)`. This new state is stored in another frame and placed on top of the previous frame in the stack.
3. This process continues until we reach `printNumbers(0)`. This state is added to the top of the stack, and because `n` is no longer greater than `0`, we don't make any further recursive calls.

The call stack now looks like this:

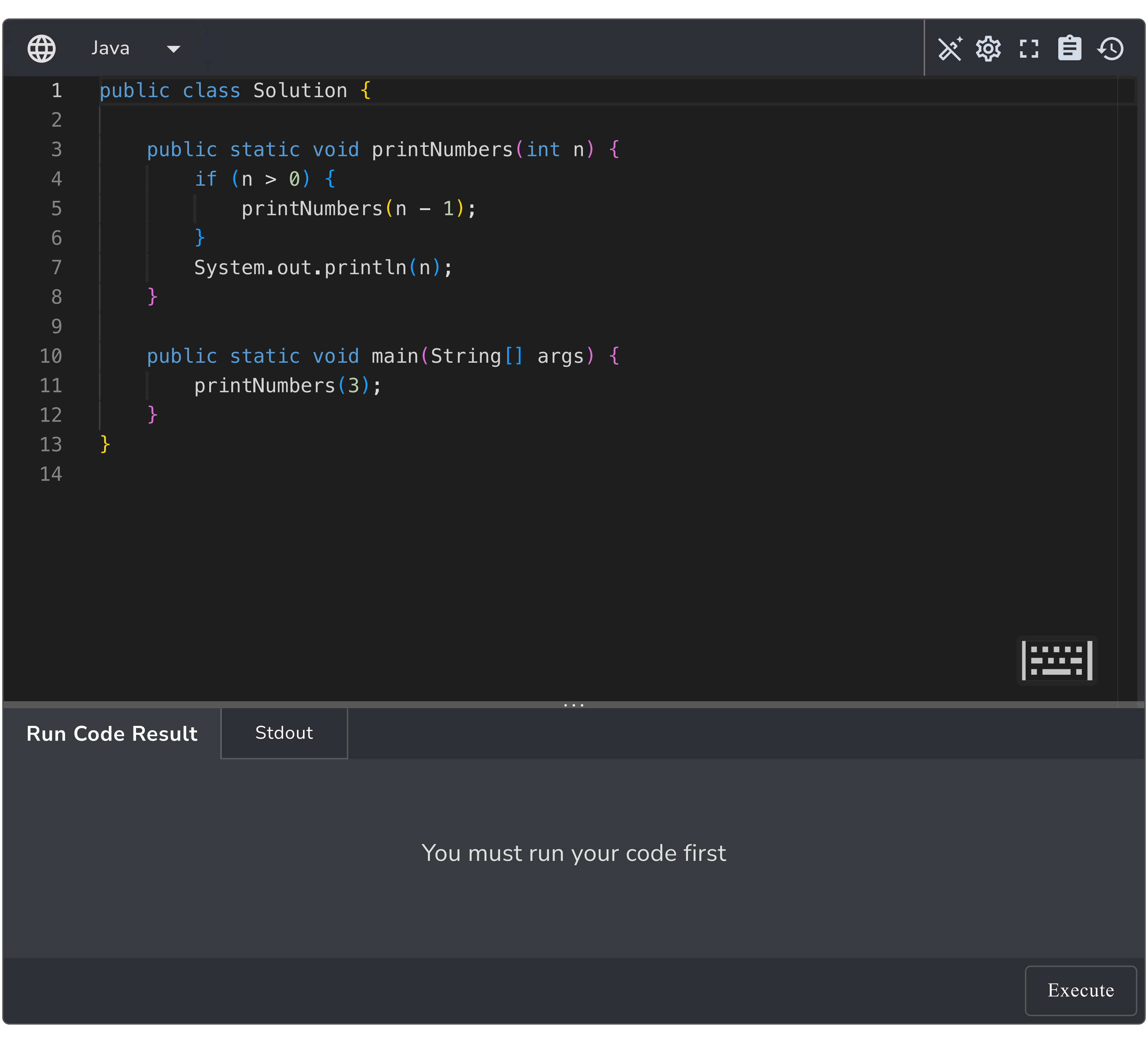


Once the recursion stops at base case, the execution control starts executing the epilogue of the recursive function `printNumbers()`. The above illustration represents the execution flow with numbered green circles.

1. The top of the stack is a call for the base case `printNumbers(0)`. Thereby, it prints `0` and gets removed from the stack.
2. We go back to calls for `printNumbers(1)`, execute `print(1)`, and remove it from the stack.
3. We repeat this process, executing `print(2)` and `print(3)` for their respective frames and removing them.

Finally, our stack gets empty leaving `0 1 2 3` on the output console. The execution control returns to the main `Driver code` for further progress with your program.

Here is the working code for this example:



```java
public class Solution {

    public static void printNumbers(int n) {
        if (n > 0) {
            printNumbers(n - 1);
        }
        System.out.println(n);
    }

    public static void main(String[] args) {
        printNumbers(3);
    }
}
```

You must run your code first

Having peeled back the curtain to expose how recursion functions under the hood with the help of call stacks and recursion tree representation, we have sufficiently "demystified" recursion. But there's more!

Up next, we will explore the different types of recursion and how some problems naturally map on these forms.